CENG 201 – Task 2 Report

Topic: Treatment Queue Implementation using Linked List

Time Complexity Analysis:

In this task, a queue data structure was implemented using a linked list to manage treatment requests in a hospital system. The queue uses two pointers: front and rear, which point to the first and last nodes respectively.

The enqueue method adds a new treatment request to the rear (end) of the queue. Since we maintain a rear pointer, we can directly access the last node without traversing the list. The operation involves creating a new node, updating the rear pointer's next reference, and updating the rear pointer itself. As a result, the time complexity of this operation is O(1).

The dequeue method removes a treatment request from the front of the queue. Since we maintain a front pointer, we can directly access the first node. The operation involves retrieving the data from the front node, updating the front pointer to the next node, and handling the case when the queue becomes empty. The time complexity of this operation is also O(1).

The isEmpty method simply checks if the front pointer is null, which has a time complexity of O(1). The size method returns the size variable that is maintained and updated with each enqueue and dequeue operation, also resulting in O(1) time complexity.

Queue vs. Stack Comparison:

A queue follows the FIFO (First In First Out) principle, where the first element added is the first one to be removed. This is suitable for treatment requests where priority patients should be processed first, but within the same priority level, requests are processed in the order they arrived. A stack, on the other hand, follows LIFO (Last In

First Out), which would not be appropriate for this scenario as it would process the most recent request first, regardless of arrival order.

In this hospital management scenario, where treatment requests need to be processed in the order they arrive (within the same priority level), a queue is the appropriate data structure choice.